Séquence 2

2

Corrigés

Le projet **projet-corrige** contient la solution complète. Les extraits ci-dessous remplacent les six zones de **frontend/src/useConnection.js** ; ils ne doivent pas être ajoutés comme un deuxième ensemble d'écouteurs.

TODO 1 — Construire la connexion

```
current = new WebSocket(url.value);
```

Retirer les deux lignes provisoires, notamment le **return**. **current** est l'objet de cette tentative, **socket** en conservera ensuite la référence. Une URL mal formée est déjà couverte par le **catch** fourni. Une adresse correcte mais indisponible nécessite encore les gestionnaires d'événements.

TODO 3 — Montrer l'ouverture

```
readyState.value = current.readyState;
information.value = 'Connexion ouverte ; les messages seront traités en S3.';
```

Le code fourni appelle ensuite **record('open').** La mise à jour de la ref déclenche celle du badge et des boutons. L'ouverture devient vérifiable dans l'interface et dans l'entrée réseau métier.

TODO 4 — Signaler l'erreur sans deviner

```
information.value = 'Connexion en erreur : vérifiez l'adresse, le serveur et l'onglet Réseau.';
```

Le **record('error', …)** existant conserve le passage par le gestionnaire. Le code n'invente pas un état 4 et ne conclut pas « serveur arrêté » pour toutes les erreurs. Le traitement de **close** termine ensuite la mise à jour visuelle.

TODO 5 — Constater la fermeture

```
readyState.value = current.readyState;
information.value = event.wasClean
  ? 'Connexion fermée. Une nouvelle ouverture reste manuelle.'
  : 'Connexion interrompue ou ouverture échouée. Vérifiez le diagnostic.';
record('close', `code=${event.code}; reason=${event.reason || '(vide)'};
wasClean=${event.wasClean}`);
```

La fermeture est enregistrée même lorsque le texte de raison est vide. L'interface autorise alors une nouvelle tentative. Le gestionnaire ne déclenche pas cette tentative lui-même.

TODO 6 — Demander une fermeture normale

```
socket.close(1000, 'Fin de la séance');
readyState.value = socket.readyState;
```

Le texte « Fermeture demandée » et l'enregistrement de journal sont déjà fournis après ces lignes. La lecture de l'état courant permet de montrer la transition ; l'état final sera constaté par **onClose**.

Trace nominale attendue

```
construction · état 0
open · état 1
close demandé · état 2
close · état 3
code=1000; reason=Fin de la séance; wasClean=true
```

Cette trace décrit le scénario nominal local avec le serveur fourni. Un incident concurrent peut changer le résultat de fermeture. La validation porte aussi sur la cohérence entre les faits observés et ce que l'interface annonce.

Pour une ouverture échouée avec une URL syntaxiquement valide, on attend une construction, un signal d'erreur et la fermeture, sans **open**. Pour une URL avec fragment, le **catch** signale l'exception avant création d'une nouvelle connexion. Les messages système et détails réseau peuvent varier selon le navigateur.

Pourquoi ces garde-fous sont-ils fournis ?

- Le test au début de **connect()** refuse une ouverture concurrente. Il reste utile même si le bouton est désactivé.
- La référence **current** appartient à une tentative. Le test **socket !== current** évite qu'un ancien callback mette à jour la nouvelle tentative.
- **removeListeners()** retire les gestionnaires précédents avant une nouvelle connexion et au démontage.
- Le nettoyage ferme une connexion encore en ouverture ou ouverte quand l'interface disparaît.

Leur écriture n'est pas évaluée aujourd'hui. Ils permettent de tester les six objectifs sans introduire une gestion complète de la reconnexion ou de l'architecture Vue.

Réponses au bilan

1. La construction lance une tentative ; **open** prouve que cette tentative a abouti.
2. **close()** demande la fermeture ; l'événement **close** signale son résultat.
3. Le signal d'erreur est générique : plusieurs causes peuvent produire le même événement. Il faut croiser adresse, disponibilité serveur et observations réseau.
4. La connexion métier vise **ws://127.0.0.1:3000/chat**, avec l'ouverture HTTP/1.1 acceptée. La connexion de développement Vite utilise le serveur frontend.